

Course & Skill Recommendation Engine: Borderless Career Accelerator

Theme: Innovate Without Borders

Item	Detail
Document type	Technical Project Documentation
Event	Global Innovation Hackathon 2026 – Build for a Better Future
Theme	Innovate Without Borders
Source of truth	Project README.md (no other project material was used)
License	Apache-2.0
Project repository	https://github.com/saifan30/global-innovation-hackathon-2026
Live demo	https://course-skill-recommendation-engine.ai.studio/
Video demo	https://youtu.be/cb8NcnQKUnw?si=qv6kcCMYeOaq0J1w
Document date	20 September 2026

This report describes the project strictly as documented. Where a detail is not present in the README, it is either omitted or marked “Not specified in the provided project documentation.” No API keys, tokens or environment values appear in this document.

Table of Contents

1. Cover Page.....	1
2. Executive Summary.....	3
3. Problem Statement.....	4
4. Proposed Solution.....	4
5. Innovation & Uniqueness.....	5
6. Target Users.....	5
7. Objectives.....	5
8. Complete 10-Step User Workflow.....	6
9. Detailed Feature Description.....	8
10. Skill-Gap Analysis Methodology.....	8
11. Recommendation Engine.....	10
12. AI Architecture.....	11
13. System Architecture.....	13
14. Technology Stack.....	15
15. API Architecture.....	15
16. Data & Source Provenance.....	16
17. Security & Privacy.....	17
18. User Experience & Accessibility.....	17
19. Testing & Quality Assurance.....	18
20. Deployment & Configuration.....	18
21. Limitations.....	19
22. Future Scope.....	20
23. Hackathon Evaluation Alignment.....	20
24. Conclusion.....	21
25. References.....	21

2. Executive Summary

Course & Skill Recommendation Engine: Borderless Career Accelerator is a full-stack web application built for the Global Innovation Hackathon 2026 under the theme *Innovate Without Borders*. It supports learners and developers who are preparing for software-engineering-related roles by comparing their self-assessed skill proficiency with a curated benchmark for a chosen target role, and by turning the result into a prioritized, pacing-calibrated learning plan.

The user moves through a linear **10-step workflow**: User Profile, Career Goal, Current Skills, Target Role Benchmark, AI Skill-Gap Diagnostics, Skill Prioritization, Personalized Learning Roadmap, Course & Resource Recommendations, Capstone Project Recommendations and Progress Tracking. Four target roles are documented: AI Engineer, Cloud/DevOps Reliability Engineer, Application Security Specialist and Data Platform Engineer.

The central design decision is a **hybrid architecture**. All numbers (readiness score, critical-skill coverage, estimated hours, recommendation scores, estimated weeks and roadmap durations) are computed by a deterministic, rule-based engine on the server. Only the strategic narrative (diagnostic summary and guidance text) is generated by Google Gemini, called server-side. If the AI service is unavailable, a rule-based fallback produces the guidance and the response is explicitly labelled as not AI-generated.

Project at a glance

Aspect	Documented fact
Target roles	4 (AI Engineer; Cloud/DevOps Reliability Engineer; Application Security Specialist; Data Platform Engineer)
Workflow	10 steps, exposed through a global workflow tracker
Proficiency scale	5 tiers, L1 (Novice) to L5 (Industry Leader)
Frontend	React 19, TypeScript 5+, Tailwind CSS 4, Motion, Lucide React
Backend	Node.js, Express 4, TypeScript (tsx in development; esbuild bundle for production)
AI	Google Gemini (g e m i n i - 3 . 8 - f l a s h) via @ g o o g l e / g e n a i (v2.4.0), server-side only
REST API	18 endpoints listed in the README (Section 15)
Persistence	In-memory server-side data store (no database)
License	Apache-2.0

Current status and honest scope. User state is held in server memory, and the learning-resource catalog covers four core engineering domains. These constraints, and others, are set out in Section 21.

3. Problem Statement

The README identifies three barriers faced by global tech talent when preparing for borderless, high-demand software-engineering careers:

#	Barrier	Description (from the README)
1	Opaque industry benchmarks	Job descriptions and syllabi often rely on generic buzzwords rather than quantifiable competency levels, leaving learners uncertain about how far their existing skills are from production standards.
2	Generic, uncurated course overload	Popular platforms present many disconnected tutorials without explaining why a resource is recommended or how many weeks it will take given the learner's specific weekly study commitment.
3	Black-box AI outputs	Many generative AI career tools can fabricate non-existent courses, invent credentials or give vague advice, with no mathematical grounding and no deterministic fallback when API limits or network outages occur.

How the project responds

Barrier	Design response in this project
Opaque benchmarks	Role benchmarks expose per-competency required levels, importance ratings (critical / important / nice-to-have) and weights; gaps are computed with published formulas (Section 10).
Course overload	Resources come from a curated catalog, are ranked by a documented scoring model, and each carries a rationale and an estimated completion time at the learner's weekly pace (Section 11).
Black-box AI	Numbers are deterministic; AI text is confined to narrative guidance, produced server-side, and a labelled rule-based fallback is used when the AI call cannot complete (Section 12).

4. Proposed Solution

The application is an end-to-end career-acceleration platform. It connects an individual competency self-assessment to standardized role benchmarks and then produces transparent skill gaps, prioritized learning areas, a multi-phase roadmap, and recommended open-access coursework and capstone projects. The README describes three pillars:

Pillar	What it does	Nature
Deterministic Calculation Engine	A rule-based weighted matrix that evaluates assessed proficiency against the role benchmark, producing readiness scores, critical gap counts and weekly study-timeline calculations.	Deterministic
Server-side AI Strategic Diagnostic	A server-side integration with Google Gemini (<code>gemini-3.8-flash</code>) that synthesizes the user's background, weekly study hours and active competency deficits into strategic advice and open-source portfolio guidance.	AI-generated (with rule-based fallback)
Explainable Recommendations	Every course and capstone recommendation shows a "Why am I seeing this?" breakdown: targeted competency deficit, required level jump, format suitability and estimated completion time.	Deterministic inputs, transparent display

Design principles evident in the documentation

- **Transparency:** formulas and point weights are documented; each recommendation is explained.
- **Separation of concerns:** quantitative results are never delegated to the AI model.
- **Graceful degradation:** the workflow continues without an AI key or when the AI call fails.
- **Open resources:** recommendations cite official documentation, open courseware and open-access guides.
- **Server-side secret handling:** the Gemini API key is only used in server code.

5. Innovation & Uniqueness

The points below describe design choices documented in the README. The README does not include a comparative survey of other tools, so no comparative or “first of its kind” claim is made here.

Innovation	Description
Hybrid deterministic + AI architecture	Objective calculations (scores, hours, roadmap durations) are computed deterministically; only strategic narrative synthesis is delegated to Gemini, with an automatic rule-based fallback.
Continuous reactivity	Changing weekly study commitment, updating a skill proficiency level or switching the target role recalculates gap metrics, roadmap milestones and recommendations.
Explainable recommendations	Each course and capstone recommendation carries a structured rationale (target competency and level jump, format alignment, pacing calibration, source provenance).
Source provenance	Recommendations cite official documentation, university open courseware (for example Harvard CS50P and University of Helsinki Full Stack Open) and open-access guides rather than unverified links.
Transparent AI labelling	Fallback responses are flagged <code>is A i G e n e r a t e d : f a l s e</code> and carry an explicit source label, so users can tell AI-generated guidance from rule-based guidance.
Built-in test personas	A persona switcher and preset endpoint let evaluators load novice, transitioning and experienced profiles to examine edge cases quickly.

6. Target Users

The README states that the platform is designed to empower talent regardless of geographic origin, institutional pedigree or socioeconomic background, with a focus on developers and learners worldwide preparing for software-engineering careers.

User group	Basis in the documentation	How the application serves them
Self-directed learners and developers worldwide	Stated focus: borderless workforce readiness for developers and learners worldwide	Free/open resources, pacing calibrated to the learner’s own weekly hours (1–80 h), no reliance on a specific institution.
Novice, transitioning and experienced profiles	Built-in test presets for these three profile types	Skill inventory on a 5-tier scale allows different starting levels; gaps and roadmap adapt to the profile.
Aspirants to four documented roles	AI Engineer; Cloud/DevOps Reliability Engineer; Application Security Specialist; Data Platform Engineer	Role benchmark, prioritized gaps, roadmap, courses and capstone projects for the selected role.
Hackathon evaluators and reviewers	Preset persona switcher described as a test aid for evaluating edge cases	Persona presets (POST /api/profile/preset) and a health endpoint that reports Gemini configuration status.

Demographic data, user research or user counts are not provided. No claims about adoption or outcomes are made.

7. Objectives

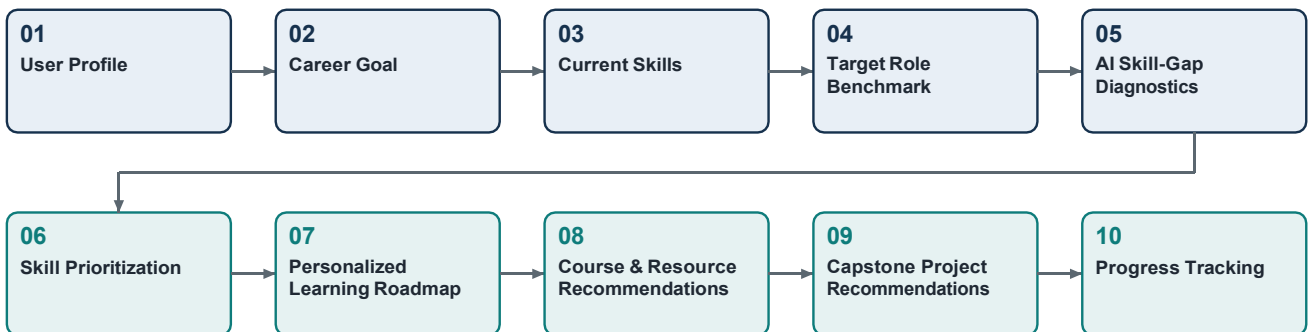
The objectives below are derived from the README’s problem statement, solution description and feature list.

#	Objective	Addressed in
1	Make the distance between a learner’s current skills and a target role measurable and transparent.	Section 10
2	Prioritize skill deficits by urgency and impact (P1–P4).	Sections 8, 11
3	Produce a multi-phase roadmap whose durations are calibrated to the learner’s weekly study hours.	Sections 8, 9

#	Objective	Addressed in
4	Recommend free/open, curated courses and portfolio-grade capstone projects, ranked by a documented scoring model.	Section 11
5	Explain every recommendation instead of presenting an opaque ranking.	Section 11
6	Use AI for strategic narrative only, keep calculations deterministic, and label the source of every insight.	Section 12
7	Keep the workflow usable when the AI service is unavailable (missing key, quota, network failure, timeout).	Section 12
8	Keep secrets on the server and validate incoming data.	Section 17
9	Let users track progress (study sessions, level-ups, phase completion, cumulative hours).	Section 8

8. Complete 10-Step User Workflow

The application implements a linear 10-stage pipeline, navigable through a global workflow tracker (`WorkflowPipeline.tsx`). The journey runs from the learner's profile through to progress tracking:



Step	Stage	UI component	Related endpoints (by purpose)
1	User Profile	<code>UserProfileView.tsx</code>	GET/PUT <code>/api/profile</code> ; POST <code>/api/profile/preset</code>
2	Career Goal	<code>CareerGoalView.tsx</code>	GET <code>/api/roles</code>
3	Current Skills	<code>CurrentSkillsView.tsx</code>	GET <code>/api/skills</code> ; POST <code>/api/profile/skills</code>
4	Target Role Benchmark	<code>RoleBenchmarkView.tsx</code>	GET <code>/api/roles/:id</code>
5	AI Skill-Gap Diagnostics	<code>SkillGapView.tsx</code>	GET <code>/api/gap-analysis</code> ; POST <code>/api/ai/strategic-insights</code>
6	Skill Prioritization	<code>SkillPrioritizationView.tsx</code>	Not specified in the README
7	Personalized Learning Roadmap	<code>RoadmapView.tsx</code>	GET <code>/api/roadmap</code> ; POST <code>/api/roadmap/generate</code> ; PUT <code>/api/roadmap/milestones/:milestoneId</code>
8	Course & Resource Recommendations	<code>CoursesView.tsx</code>	GET <code>/api/recommendations/courses</code>
9	Capstone Project Recommendations	<code>ProjectsView.tsx</code>	GET <code>/api/recommendations/projects</code>
10	Progress Tracking	<code>ProgressTrackerView.tsx</code>	GET <code>/api/progress</code> ; POST <code>/api/progress/log</code> ; PUT <code>/api/roadmap/milestones/:milestoneId</code>

Endpoint associations are matched by the purposes listed in the README's API table; the README does not provide an explicit step-to-endpoint mapping. A summary dashboard (`DashboardOverview.tsx`) presents readiness analytics.

Step 1 – User Profile

The learner configures their background, weekly learning hours (documented range 1–80 h), learning pace, preferred learning formats (interactive labs, project-based, video, documentation) and borderless career aspirations. These inputs later drive pacing calculations and format matching in the recommendation model. Profile updates are sanitized, type-validated and clamped on the server.

Step 2 – Career Goal

The learner selects a target industry role from the four documented roles and reviews market-demand metrics. The README does not state where the market-demand values originate or how they are maintained.

Step 3 – Current Skills Inventory

The learner completes an interactive self-assessment across core technical competencies, frameworks and architecture patterns, rating each on a 5-tier proficiency scale from L1 (Novice) to L5 (Industry Leader). Skill assessments can be batch-updated or upserted through the API. Because ratings are self-reported, they are inputs to, not verified results of, the analysis.

Step 4 – Target Role Benchmark

The learner inspects the curated curriculum benchmark for the selected role: competency importance ratings (critical, important, nice-to-have), benchmark weights, and the industry-standard sources behind the benchmark. These required levels and weights are the reference for the gap calculation.

Step 5 – AI Skill-Gap Diagnostics

The learner sees a visual 5-tier proficiency comparison matrix (current vs. required) together with strategic insights. The matrix values come from the deterministic gap analysis; the insights come from Gemini when available, or from the deterministic diagnostic engine otherwise. The step is titled “AI” because of the Gemini narrative; the underlying numbers are not AI-generated.

Step 6 – Skill Prioritization

Competency deficits are mapped into an urgency-and-impact matrix with four levels: **P1 Immediate**, **P2 Core Next**, **P3 Specialization** and **P4 Polish**. The point weights attached to these levels in the recommendation model are documented (Section 11); the exact rule that assigns a given gap to a level is *not specified in the provided project documentation*.

Step 7 – Personalized Learning Roadmap

The learner follows a multi-phase milestone curriculum (Foundation, Specialization, Capstone) with phase objectives, phase durations in weeks calibrated to weekly study hours, and completion checkpoints. The roadmap can be regenerated when the profile changes, and milestone progress percentage and completion status can be updated.

Step 8 – Course & Resource Recommendations

Curated free and open educational resources are ranked by match score. Each is shown with an explainability card and a syllabus citation (publisher, accreditation and verified syllabus URL, as described in the README).

Step 9 – Capstone Project Recommendations

The learner selects real-world, portfolio-grade capstone projects. Each recommendation includes a suggested production technology stack, a skill-gain mapping and guidance for open-source submission.

Step 10 – Progress Tracking

The learner logs completed study sessions, records skill level-ups, monitors phase-completion percentages and views cumulative hours invested. Log entries are validated on the server. Whether recorded level-ups automatically update the skill inventory used by the gap analysis is *not specified in the provided project documentation*.

9. Detailed Feature Description

The following table lists each feature documented in the README, with its behavior and the relevant implementation location where the README names one.

Feature	Behavior as documented	Location / notes
Interactive 5-tier proficiency matrix	High-contrast visual comparison blocks (L1 to L5) indicating acquired competencies, critical deficits and partial gaps.	<code>SkillGapView.tsx</code>
Dynamic study pacing	Estimated completion weeks are recalculated automatically when the weekly study commitment is adjusted.	Formula in Section 11
Full data reactivity	Updates to user skills or the target role cascade across gap analysis, roadmaps and course scoring.	<code>AppContext.tsx</code> synchronizes client state with the API
Structured rationale breakdown	Every recommendation states the reasons for its ranking, including gap coverage, format matching and pacing metrics.	<code>CoursesView.tsx</code> , <code>ProjectsView.tsx</code>
Verified open educational resources library	Curated library linking to free, open-access and accredited materials, with source citations (Section 16).	<code>server/db/data.ts</code>
Persona switcher	Test presets covering novice, transitioning and experienced profiles for evaluating edge cases.	<code>presets.ts</code> ; <code>POST /api/profile/preset</code>
Hybrid AI insights with fallback	Gemini strategic diagnostics with an 8-second timeout guard and automatic rule-based fallback; source is labelled.	<code>insightsService.ts</code>
Insight caching	In-memory cache with a 60-second TTL, keyed by profile parameters and invalidated on skill or preference updates.	Server memory
Roadmap generation and milestones	Multi-phase roadmap (Foundation, Specialization, Capstone) with milestone progress/completion updates.	<code>RoadmapView.tsx</code> ; <code>DataStore</code>
Progress tracker	Logs study sessions and skill level-ups; shows phase completion percentages and cumulative hours.	<code>ProgressTrackerView.tsx</code>
Global workflow tracker and dashboard	10-step navigation bar and a summary dashboard with readiness analytics.	<code>WorkflowPipeline.tsx</code> , <code>DashboardOverview.tsx</code>
Notifications	Toast alerts for user actions and status.	<code>NotificationToaster.tsx</code>

10. Skill-Gap Analysis Methodology

All skill-gap mathematics is performed **deterministically on the server** in `DataStore.computeSkillGapAnalysis()`. No AI model participates in these calculations.

10.1 Inputs

- **Current level** per competency: the learner's self-assessed proficiency on the 5-tier scale (L1 Novice to L5 Industry Leader).
- **Required level** per competency: the benchmark level for the selected target role.
- **Weight** per competency: the benchmark weight shown in the Target Role Benchmark step.
- **Importance** per competency: critical, important or nice-to-have.

The README documents the endpoints of the scale (L1 = Novice, L5 = Industry Leader). The names of levels L2 to L4 and the numerical weight values are *not specified in the provided project documentation*. The README places the roles and skills dataset in `server/db/data.ts`.

10.2 Comparing current skills with target-role requirements

For every competency in the role benchmark, the engine compares the learner's current level with the required level. The comparison yields the level the learner has already acquired (capped at the requirement) and the remaining gap (never negative).

Per-competency calculations

Acquired Level = $\min(\text{CurrentLevel}, \text{RequiredLevel})$

Gap Level = $\max(0, \text{RequiredLevel} - \text{CurrentLevel})$

10.3 Aggregate metrics

Weighted points

Weighted Acquired Points = $\sum (\text{Acquired Level} \times \text{Weight})$

Weighted Total Benchmark Points = $\sum (\text{Required Level} \times \text{Weight})$

Overall Readiness Score

Readiness Score = $\text{round}(\text{Weighted Acquired Points} \div \text{Weighted Total Benchmark Points} \times 100)$

Critical Skills Coverage

Critical Coverage = $\text{round}(\text{Mastered Critical Skills} \div \text{Total Critical Skills} \times 100)$

Estimated Hours to Close the Gap

Hours = $\sum (\text{GapLevel} \times H)$, where $H = 25$ if the skill is critical, otherwise $H = 18$

10.4 Interpretation and worked arithmetic

The README's own example of a level jump is "Level 1 $\rightarrow$ Level 4". Applying the documented formulas to that jump: acquired level = $\min(1, 4) = 1$ and gap level = $\max(0, 4 - 1) = 3$. The estimated hours for that single skill would be $3 \times 25 = 75$ hours if the skill is critical, or $3 \times 18 = 54$ hours otherwise. This is arithmetic on the documented constants, not a measured result.

Not specified in the provided project documentation

The definition of a "Mastered Critical Skill" (for the Critical Coverage numerator), the handling of an empty denominator, and the numerical benchmark weights are not stated in the README. This document therefore does not define them.

10.5 How the results are used

- The **proficiency matrix** (Step 5) visualizes current vs. required levels; acquired competencies, critical deficits and partial gaps are shown with high-contrast blocks.
- The **readiness score**, critical gap list and evaluated timeline metrics form part of the context supplied to Gemini (Section 12).
- **Gap levels and critical gaps** feed the recommendation scoring (Section 11).

11. Recommendation Engine

Courses/resources and capstone projects are ranked by a **deterministic multi-factor scoring model**. There is no AI model in the scoring path; the same profile and catalog produce the same scores.

11.1 Scoring factors

Factor	Documented rule	Points
Deficit mapping	Evaluates whether the resource addresses active skill gaps for the selected target role.	Determines eligibility for the factors below
Priority weighting	P1 Critical Deficit	+70 (base)
	P2 Core Next Deficit	+45 (base)
	P3 Specialization	+25 (base)
	P4 Mastery Polish	+10 (base)
Gap multipliers	Per critical gap addressed	+20 per gap
	Per active gap covered	+15 per gap
Format alignment	Resource matches the learner's preferred learning formats (for example interactive labs, project-based work).	+20
Resource quality	Resource rating metric.	+3 × rating

The README lists these contributions as additive point awards. It does not give a single combined score formula, does not say which priority level applies when a resource spans several gaps, and does not describe tie-breaking. These details are *not specified in the provided project documentation*.

11.2 Pacing calculation

Estimated completion time at the learner's pace

Estimated Weeks = $\max(0.5, \text{Resource Estimated Hours} \div \text{Weekly Study Hours})$

Because Weekly Study Hours comes from the profile (documented range 1–80 h), changing the weekly commitment changes the displayed weeks immediately.

11.3 How personalization uses documented inputs

Input	How it is used (as documented)
Target role	Defines the benchmark and therefore which competencies are gaps; resources are evaluated against the active gaps for this role.
Current skills and proficiency	Determine gap levels through the gap analysis; the required level jump (for example Level 1 → Level 4) is displayed in the rationale.
Gaps (critical and active)	Add +20 per critical gap addressed and +15 per active gap covered.
Priorities (P1–P4)	Provide base points of 70, 45, 25 and 10 respectively.
Preferred formats (modality)	+20 points when the resource matches the learner's preferred formats.
Weekly learning hours	Divide the resource's estimated hours to obtain estimated weeks; also shape roadmap phase durations.
Resource rating	Adds 3 × rating.
Logged progress	The README states that skill updates cascade to course scoring. Whether logged progress records or milestone status directly change scores is <i>not specified</i> .

11.4 Course/resource and capstone project recommendations

The README describes one scoring model for both resources and capstone projects. Course recommendations are returned by `GET /api/recommendations/courses` and capstone recommendations by `GET /api/recommendations/projects`. Capstone recommendations additionally present a suggested production tech stack, a skill-gain mapping and open-source submission guidance. Any project-specific weights, if they exist, are *not specified in the provided project documentation*.

11.5 Why explainability is provided

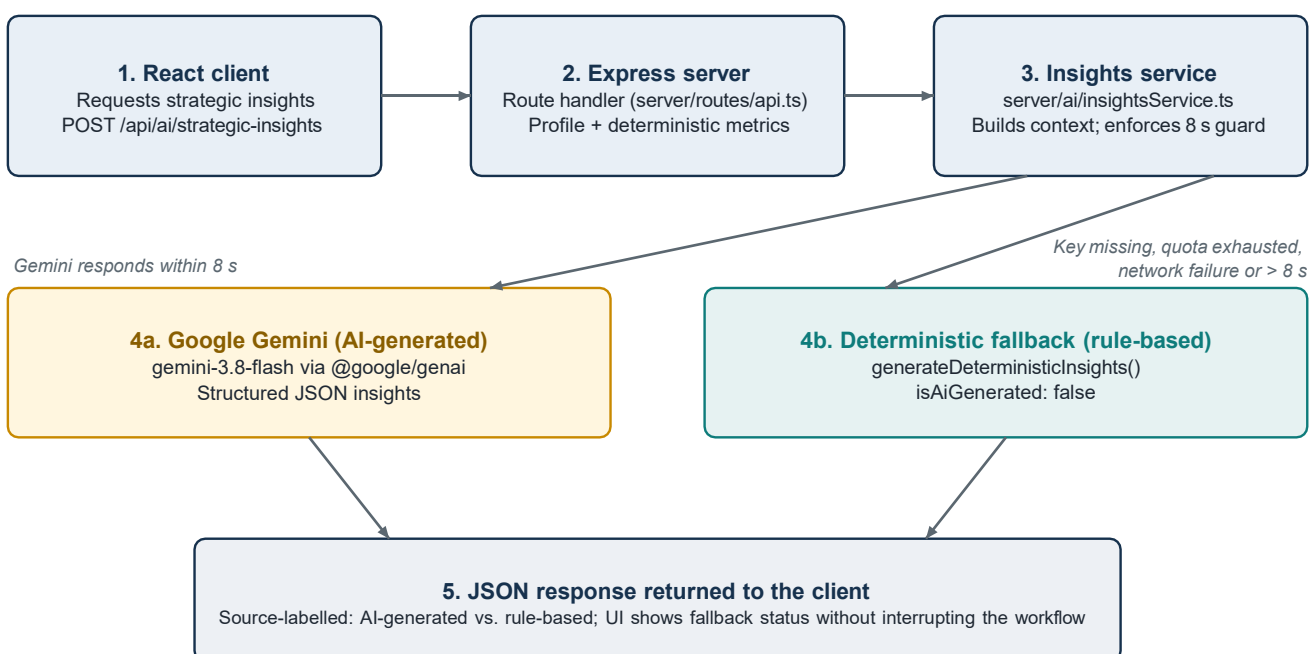
The README positions explainability as a response to black-box algorithmic recommendations. Each recommendation displays a “Why am I seeing this?” rationale so learners can judge whether a resource fits, and evaluators can verify that ranking follows the documented rules.

Rationale element	What the user sees
Target competency	The exact skill and level jump (for example Level 1 → Level 4).
Format alignment	Whether the medium (for example interactive lab or video series) matches the profile’s preferred formats.
Pacing calibration	Total estimated resource hours and expected weeks at the user’s declared weekly pace.
Source provenance	Publisher, accreditation and verified syllabus URL.

12. AI Architecture

12.1 Client → backend → AI service flow

Gemini is called exclusively from the Express backend. The React client calls the Express backend (`POST /api/ai/strategic-insights`, or its alias `POST /api/gap-analysis/ai-insights`); the backend builds the AI context from server-held data and calls Gemini through the `@google/genai` SDK. If the call cannot complete, the backend answers with the deterministic fallback.



12.2 Server-side API-key handling

- The `GEMINI_API_KEY` environment variable is loaded strictly in server-side code (`server.ts`, `server/gemini.ts`, `server/ai/insightsService.ts`).
- No API keys or tokens are passed to, or bundled in, client-side code.

- The SDK is initialized lazily in `server/gemini.ts`; the key is optional, and the system falls back to the deterministic engine if it is omitted.
- `GET /api/health` reports Gemini configuration status (the README does not specify the response format).

12.3 Context supplied and output required

The server supplies Gemini with verified user-profile data, weekly hours, learning preferences, target-role descriptions, the readiness score, critical gap lists and evaluated timeline metrics. Prompt constraints require Gemini to return structured JSON with the following parts:

- Executive diagnostic summary
- Key strategic directives
- Priority focus areas
- Weekly pace allocation guidance
- Borderless innovation and open-source portfolio strategy
- Capstone project execution guidance

The exact prompt text and any post-generation validation of the model's JSON are *not specified in the provided project documentation*.

12.4 Timeout, cache and fallback behavior

Mechanism	Documented behavior
Timeout protection	AI requests enforce an 8-second timeout guard (<code>P r o m i s e . r a c e</code>) to prevent network hanging.
In-memory cache	Insights are cached with a 60-second TTL, keyed by profile parameters, and invalidated when skills or preferences are updated.
Fallback triggers	Missing Gemini API key; exhausted API quota; network connectivity failure; request exceeding the 8-second threshold.
Fallback action	The server activates <code>generateDeterministicInsights()</code> , a rule-based diagnostic that turns the user's verified metrics into structured guidance.
Fallback labelling	The response is flagged <code>isAiGenerated: false</code> with the source label "Deterministic Competency Diagnostic Engine (Rule-Based Engine)".
User-visible effect	The interface reflects the fallback status transparently, without errors, crashes or interrupted workflows (per README).

The order of cache lookup relative to the Gemini call, the retry policy (if any) and any rate limiting are *not specified in the provided project documentation*.

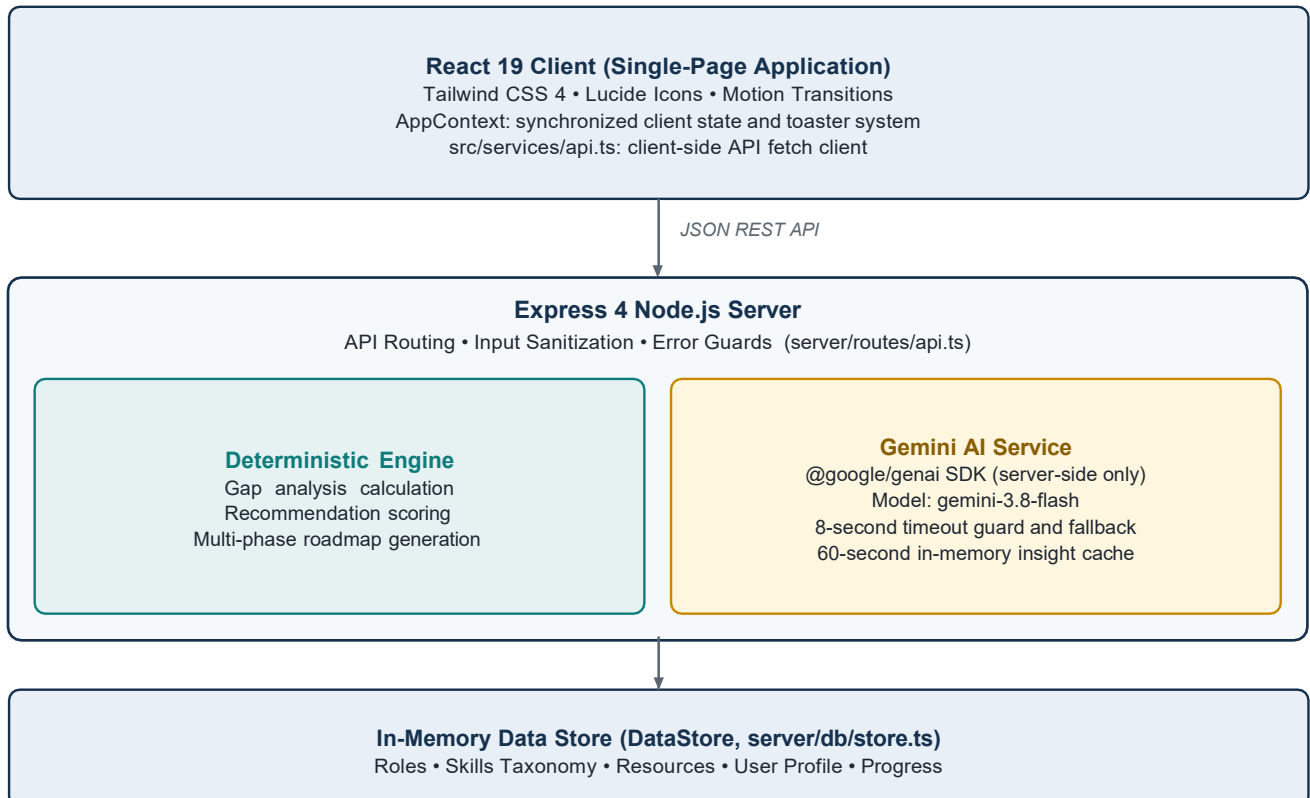
12.5 AI-generated vs. deterministic outputs

Output	Produced by	Type
Acquired level, gap level, weighted points	<code>DataStore.computeSkillGapAnalysis()</code>	Deterministic
Readiness score, critical coverage, estimated hours to close	<code>DataStore.computeSkillGapAnalysis()</code>	Deterministic
Recommendation scores and ranking (courses, projects)	Deterministic multi-factor scoring model	Deterministic
Estimated weeks per resource	Pacing formula (Section 11.2)	Deterministic
Multi-phase roadmap generation	Deterministic engine (per system diagram)	Deterministic
Rationale ("Why am I seeing this?") content	Derived from documented scoring inputs and catalog data	Deterministic
Strategic insight text: summary, directives, focus areas, pace guidance, portfolio strategy, capstone guidance	Google Gemini via server	AI-generated
Fallback insight text (when AI unavailable)	<code>generateDeterministicInsights()</code>	Rule-based (<code>isAiGenerated: false</code>)

The README does not describe AI involvement in any other output.

13. System Architecture

The system is a single-page React client that talks to an Express server over a JSON REST API. The server hosts the deterministic engine, the Gemini AI service and an in-memory data store. The diagram below reflects the component diagram in the README.



13.1 Component responsibilities

Layer	Component	Responsibility (as documented)
Client	<code>src/context/AppContext.tsx</code>	Global state manager and API synchronization; toaster system.
Client	<code>src/services/api.ts</code>	Client-side API fetch client.
Client	<code>src/components/views/*</code>	One view per workflow step plus a dashboard overview.
Server	<code>server.ts</code>	Express server entry point and Vite middleware.
Server	<code>server/routes/api.ts</code>	Express REST API route handlers (routing, input sanitization, error guards).
Server	<code>server/db/store.ts</code>	DataStore: gap calculations, roadmaps and scoring.
Server	<code>server/db/data.ts</code>	Roles, skills taxonomy, courses and projects.
Server	<code>server/db/presets.ts</code>	Test personas for evaluation.
Server	<code>server/ai/insightsService.ts</code>	Gemini client and deterministic fallback.
Server	<code>server/gemini.ts</code>	Lazy initialization of the <code>@google/genai</code> SDK.

13.2 Data flow

- The learner interacts with a step view; **AppContext** keeps client state synchronized with the API.
- The API client sends JSON requests to Express routes; incoming payloads for profile, skills and progress-log writes are sanitized and validated.
- **DataStore** computes gap analysis, roadmaps and recommendation scores from in-memory roles, skills, resources and the user profile.
- For strategic insights, the insights service supplies deterministic metrics and profile context to Gemini, or falls back to the rule-based diagnostic.
- JSON responses return to the client, which updates the views; notifications are shown through the toaster.

13.3 Project structure

```

.env.example           # Environment variable declarations
metadata.json         # Application metadata and permissions
package.json          # Project scripts and dependencies
server.ts             # Express server entry point & Vite middleware
tsconfig.json         # TypeScript compiler configuration
vite.config.ts        # Vite configuration with Tailwind CSS plugin
server/
  ai/insightsService.ts # Gemini client & deterministic fallback
  db/data.ts           # Roles, skills taxonomy, courses & projects
  db/presets.ts        # Test personas for evaluation
  db/store.ts          # DataStore: gap calculations, roadmaps & scoring
  gemini.ts            # Lazy initialization of @google/genai SDK
  routes/api.ts        # Express REST API route handlers
src/
  App.tsx             # App root component and workflow pipeline view
  main.tsx            # React client entry point
  index.css            # Tailwind CSS imports
  context/AppContext.tsx # Global state manager & API synchronization
  services/api.ts      # Client-side API fetch client
  types/index.ts       # Shared TypeScript interfaces and types
  components/layout/   # Header, WorkflowPipeline, Footer, NotificationToaster
  components/views/    # DashboardOverview + 10 step views (Steps 1-10)

```

14. Technology Stack

The stack below is reproduced from the README. Version numbers appear only where the README states them; the Node.js version is not stated.

Layer	Technology	Version (as documented)	Role
Frontend	React	19	Single-page application UI
Frontend	TypeScript	5+	Typed client code
Frontend	Tailwind CSS	4	Styling (Vite plugin configured in vite.config.ts)
Frontend	Motion	Not specified	Animated transitions
Frontend	Lucide React	Not specified	Icons
Backend	Node.js	Not specified	Server runtime
Backend	Express	4	REST API server; hosts Vite middleware in development
Backend	tsx	Not specified	TypeScript runtime in development
Backend	esbuild	0.25	Production server bundle (dist/server.cjs)
AI	@google/genai	v2.4.0	SDK for Google Gemini, server-side only
AI	Google Gemini	gemini-3.8-flash	Strategic insight generation
Build	Vite	8	Client build (dist/) and development middleware
Data	In-memory DataStore	n/a	Roles, skills taxonomy, resources, user profile, progress (no database)

Documented npm scripts: `npm run dev`, `npm run build`, `npm start` and `npm run lint` (which runs `tsc --noEmit`). The model identifier and SDK version are reproduced exactly as written in the README.

15. API Architecture

The backend exposes a JSON REST API implemented in `server/routes/api.ts`. The table lists **only** the endpoints in the README (18 in total). Request and response schemas are not documented in the README and are therefore not described here.

Method	Endpoint	Purpose (as documented)
GET	<code>/api/health</code>	Service health status, hackathon metadata and Gemini configuration status
GET	<code>/api/roles</code>	Retrieve all available industry benchmark roles
GET	<code>/api/roles/:id</code>	Retrieve detailed benchmark requirements for a specific role
GET	<code>/api/skills</code>	Retrieve the global competency and skills taxonomy
GET	<code>/api/profile</code>	Retrieve the active user profile and skill assessments
PUT	<code>/api/profile</code>	Update user profile fields (hours, pace, formats, aspirations)
POST	<code>/api/profile/skills</code>	Batch update or upsert individual skill assessments
POST	<code>/api/profile/preset</code>	Load a test persona preset for evaluation
GET	<code>/api/gap-analysis</code>	Calculate the mathematical skill-gap analysis for the target role
POST	<code>/api/ai/strategic-insights</code>	Generate Gemini strategic diagnostics with fallback
POST	<code>/api/gap-analysis/ai-insights</code>	Alias endpoint for AI strategic insights

Method	Endpoint	Purpose (as documented)
GET	<code>/api/roadmap</code>	Retrieve the pacing-calibrated learning roadmap
POST	<code>/api/roadmap/generate</code>	Regenerate roadmap milestones based on the updated profile
PUT	<code>/api/roadmap/milestones/:milestoneId</code>	Update progress percentage and completion status for a milestone
GET	<code>/api/progress</code>	Retrieve logged progress records and study history
POST	<code>/api/progress/log</code>	Create a new study session or milestone completion log
GET	<code>/api/recommendations/courses</code>	Retrieve personalized, scored course recommendations
GET	<code>/api/recommendations/projects</code>	Retrieve personalized capstone project recommendations

Notes on API behavior

- Write endpoints `PUT /api/profile`, `POST /api/profile/skills` and `POST /api/progress/log` sanitize, type-validate and clamp incoming payloads.
- The AI insights endpoint and its alias are documented as returning either Gemini output or the deterministic fallback.
- Authentication, pagination, versioning, error-code conventions and rate limits are *not specified in the provided project documentation*.

16. Data & Source Provenance

The role benchmarks, skills taxonomy, courses and capstone projects are held in `server/db/data.ts`. The README states that benchmarks and open learning resources cite official curricula. The table shows the sources named for each domain.

Domain	Source named in the README	URL (as given)
Generative AI Engineering	DeepLearning.AI; Google Cloud AI Education	https://www.deeplearning.ai/
Cloud & Site Reliability Engineering	Google SRE Handbook; CNCF Curriculum (URL for CNCF not given)	https://sre.google/sre-book/table-of-contents/
Application Security	OWASP Foundation Top 10; PortSwigger Web Security Academy	https://owasp.org/www-project-top-ten/ https://portswigger.net/web-security
Data Platform Engineering	Google Cloud Professional Data Engineer Guide	https://cloud.google.com/learn/certification/data-engineer
Python Foundations	Harvard University CS50P	https://cs50.harvard.edu/python/
Full-Stack Engineering	University of Helsinki Full Stack Open	https://fullstackopen.com/en/
TypeScript	Microsoft TypeScript Handbook	https://www.typescriptlang.org/docs/handbook/intro.html
Systems & Concurrency	The Go Tour; The Rust Book	https://go.dev/tour/ https://doc.rust-lang.org/book/
Distributed Systems Architecture	The System Design Primer	https://github.com/donnemartin/system-design-primer

Provenance handling in the application

- Each recommendation's rationale includes source provenance: publisher, accreditation and the syllabus URL.
- External resource links use `target="_blank" with rel="noreferrer noopener"`.

- The library is described as linking only to free, open-access and accredited materials.

Provenance limits

- The README describes the role benchmarks as “audited”. No independent third-party audit, methodology or reviewer is documented, so this report describes them as curated benchmark data.
- Individual benchmark weights, required levels and market-demand values are not listed in the README.
- The README does not describe an automated process for checking link availability or resource currency.
- User data (profile, skills, progress) is created by the user during a session and is kept in server memory (Section 17).

17. Security & Privacy

Only measures explicitly supported by the README are listed as implemented.

Measure	Documented implementation
Server-side secret handling	<code>GEMINI_API_KEY</code> is loaded only in <code>server.ts</code> , <code>server/gemini.ts</code> and <code>server/ai/insightsService.ts</code> . No keys or tokens are passed to, or bundled in, client-side code.
Input sanitization	Incoming payloads for <code>PUT /api/profile</code> , <code>POST /api/profile/skills</code> and <code>POST /api/progress/log</code> are sanitized, type-validated and clamped to prevent malformed data.
Safe external links	Outbound resource links use <code>target="_blank"</code> together with <code>rel="noreferrer noopener"</code> .
Secret hygiene in configuration	<code>A.env.example</code> declares variables; the README instructs that actual secrets and <code>.env</code> files must never be committed.
Optional AI key	The application runs without a Gemini key, using the deterministic engine.
Timeout guard	An 8-second guard prevents the AI call from hanging the request (availability control).

Privacy-relevant facts from the documentation

- **Data sent to the AI provider.** When a key is configured, the server supplies Gemini with profile data, weekly hours, learning preferences, target-role descriptions, the readiness score, critical gap lists and timeline metrics.
- **Storage.** Profile, roadmap and progress data are held in server memory (DataStore) for demonstration; the README does not describe database storage, retention periods or deletion procedures.
- **Single active profile.** The API retrieves “the active user profile”; user accounts and per-user isolation are not described.

Not specified in the provided project documentation

Authentication and authorization, rate limiting, CORS policy, transport encryption (HTTPS) configuration, security headers, dependency scanning, penetration testing and any privacy policy or consent flow are not documented in the README. They are not claimed here.

18. User Experience & Accessibility

18.1 Experience design (as documented)

- **Guided linear journey** through 10 steps with a global workflow tracker.
- **Header with persona switcher** for testing novice, transitioning and experienced profiles.
- **Dashboard overview** with summary analytics and readiness.
- **Visual proficiency matrix** using high-contrast blocks for acquired, partial and critical gaps.
- **Immediate feedback:** changes to skills, weekly hours or role update metrics, roadmap and recommendations.
- **Explainability cards** on each recommendation.

- **Transparent fallback:** AI unavailability is shown as a status, not as an error.
- **Toast notifications** and motion transitions (Motion library); Lucide icons; Tailwind CSS styling.

18.2 Accessibility

The README documents high-contrast visual comparison blocks in the proficiency matrix and safe link attributes. It makes no claim of WCAG conformance, and it does not document an accessibility audit, keyboard-navigation testing, screen-reader testing or responsive/mobile testing. Accordingly, this report makes no accessibility-compliance claim.

Usability testing with real users is *not specified in the provided project documentation*.

19. Testing & Quality Assurance

The README states that the application underwent a “comprehensive 20-point quality assurance validation” and itemizes five verification areas. Only those items are reproduced below; the remaining points of the 20 are not itemized.

Verification area	Method (as documented)	Reported outcome (as documented)
Static type checking	<code>npm run lint (tsc --noEmit)</code>	Verified with zero errors
Production compilation	<code>npm run build</code>	Verified, with complete asset bundling
End-to-end user flow	Walkthrough of all 10 pipeline steps, from profile setup through progress logging	Validated
Reactivity	Changing skill proficiency or weekly hours	Confirmed that readiness scores and roadmap duration update immediately
Deterministic fallback	Tested AI failure states	Confirmed fallback to rule-based diagnostics without errors

Not documented: automated unit or integration test suites, test counts, pass rates, code-coverage figures, load or performance testing, cross-browser testing and security testing. No such numbers are stated in this report.

20. Deployment & Configuration

20.1 Installation

```
git clone [Project Repository URL - to be added]
cd [repository-folder]
npm install
```

20.2 Environment variables

Create a `.env` file in the project root by copying `.env.example` (`cp .env.example .env`). Never commit real secrets or `.env` files.

Variable	Required?	Purpose (as documented)
<code>GEMINI_API_KEY</code>	Optional	Google Gemini API key; if omitted, the system falls back to the deterministic engine. (Value intentionally not shown.)
<code>APP_URL</code>	Optional in local development	Deployment or preview URL for self-referential links.

20.3 Running and building

Task	Command	Result (as documented)
Development	<code>npm run dev</code>	Runs the full-stack Express server with Vite middleware; binds to host 0.0.0.0, port 3000; open http://localhost:3000
Production build	<code>npm run build</code>	Compiles client assets with Vite into <code>dist/</code> and a standalone server bundle with esbuild into <code>dist/server.cjs</code>
Production start	<code>npm start</code>	Starts the production server
Type check	<code>npm run lint</code>	Runs <code>tsc --noEmit</code>

The hosting platform, container/CI configuration, Node.js version and any deployed URL are *not specified in the provided project documentation*. Deployed instance details are also pending (live demo URL “to be added”).

21. Limitations

The README lists two limitations. The additional items below follow directly from documented design or from information the README does not provide.

Limitation	Detail	Source
In-memory persistence	User profile state, roadmaps and progress logs are managed in server memory (DataStore) for session demonstration. Persistent cloud storage such as Firestore or PostgreSQL would be needed for multi-tenant production deployments.	README
Curated catalog scope	The learning-resource library contains curated resources for 4 core engineering domains; additional domains require expanding the catalog dataset.	README
Self-reported proficiency	Skill levels are entered by the learner via self-assessment; automated verification is only proposed as future scope.	Derived from Steps 3 and Future Scope
Single active profile; no accounts	The API exposes an “active user profile”; user authentication is listed as future work.	Derived from README
AI output is not deterministic	Gemini-generated narrative can vary and is subject to service availability, quotas and network conditions; fallback covers failures but not content quality. Output validation beyond the JSON structure requirement is not described.	Derived from README
Benchmark provenance detail	Benchmarks are described as “audited” but no external audit is documented; weights and required levels are not listed in the README.	Derived from README
Undocumented scoring details	Aggregation of the point factors, priority assignment rules and definition of “mastered” critical skills are not specified.	README silent
Testing evidence	Five verification areas are itemized; no automated test-suite results or coverage figures are documented.	README
Assets pending	Repository, live demo and video demo URLs are placeholders.	README

22. Future Scope

The README proposes four directions. They are plans, not implemented features.

Direction	Description (from the README)
Cloud database persistence	Migrate from in-memory storage to managed cloud databases with user authentication.
Automated assessment quizzes	Interactive coding challenges that verify claimed proficiency levels objectively.
Direct GitHub integration	Automated analysis of a user's GitHub repositories to detect demonstrated competencies.
Global peer study circles	Connect borderless learners working on the same roadmap phase for collaborative capstone development.

Each direction addresses a limitation in Section 21: persistence and accounts, self-reported proficiency, and the single-user demonstration scope.

23. Hackathon Evaluation Alignment

The table maps the project, factually, to the six evaluation criteria. It records documented evidence and caveats. It does not assign a score and makes no prediction of outcome.

Criterion	Documented evidence	Caveats / not documented
Innovation & Originality	Hybrid deterministic + AI design with explicit source labelling; per-recommendation explainability; continuous reactivity; provenance-focused resource library.	No comparative market analysis in the README.
Technical Implementation	Full-stack TypeScript (React 19, Express 4, Vite 8); 18 REST endpoints; server-side Gemini via @google/genai with 8 s timeout, 60 s cache and fallback; deterministic gap and scoring formulas; input sanitization; production bundle via esbuild.	In-memory storage; no automated test-suite results documented.
Real-World Impact	Targets the three documented barriers (opaque benchmarks, course overload, black-box AI); free/open resources; four roles; borderless focus.	No user study or outcome data. No employment or learning-outcome claims are made.
Feasibility & Scalability	Simple setup (npm install, npm run dev); key optional; runs without AI; documented migration path to cloud database with authentication.	Scalability and load testing not documented; multi-tenant use requires persistence.
UX & Design	10-step tracker, dashboard, visual matrix, explainability cards, toast notifications, persona switcher, transparent fallback status.	No usability study or accessibility audit documented.
Presentation & Demonstration	Persona presets support quick demonstrations across profile types; end-to-end 10-step flow reported as validated; this report.	Live demo and video demo URLs are placeholders ("to be added").

24. Conclusion

Course & Skill Recommendation Engine: Borderless Career Accelerator turns a self-assessment and a target role into a measurable gap analysis, a prioritized skill list, a pacing-calibrated roadmap and explained resource and project recommendations. Its distinguishing choice is to keep every number deterministic and documented while confining AI to narrative guidance that is produced server-side, cached briefly, guarded by a timeout and replaced by a labelled rule-based fallback when needed.

The project is presented as a working demonstration with clearly stated boundaries: session-level in-memory state, a catalog spanning four engineering domains and self-reported proficiency. The documented future scope (cloud persistence with authentication, verified assessments, GitHub-based competency detection and peer study circles) addresses those boundaries directly.

25. References

Only resources explicitly mentioned in the README are listed. The README cites no peer-reviewed academic publications.

A. Project material

#	Reference
[1]	Course & Skill Recommendation Engine: Borderless Career Accelerator, project README.md (primary source for this document). License: Apache-2.0.

B. Learning and curriculum sources named in the README

#	Source	URL
[2]	DeepLearning.AI	https://www.deeplearning.ai/
[3]	Google Cloud AI Education	<i>URL not given in the README</i>
[4]	Google SRE Handbook	https://sre.google/sre-book/table-of-contents/
[5]	CNCF Curriculum	<i>URL not given in the README</i>
[6]	OWASP Foundation Top 10	https://owasp.org/www-project-top-ten/
[7]	PortSwigger Web Security Academy	https://portswigger.net/web-security
[8]	Google Cloud Professional Data Engineer Guide	https://cloud.google.com/learn/certification/data-engineer
[9]	Harvard University CS50P (CS50's Introduction to Programming with Python)	https://cs50.harvard.edu/python/
[10]	University of Helsinki, Full Stack Open	https://fullstackopen.com/en/
[11]	Microsoft TypeScript Handbook	https://www.typescriptlang.org/docs/handbook/intro.html
[12]	The Go Tour	https://go.dev/tour/
[13]	The Rust Book	https://doc.rust-lang.org/book/
[14]	The System Design Primer	https://github.com/donnemartin/system-design-primer

C. Frameworks, libraries and services named in the README

React 19; TypeScript 5+; Tailwind CSS 4; Motion; Lucide React; Node.js; Express 4; tsx; esbuild 0.25; Vite 8; @google/genai (v2.4.0); Google Gemini (gemini-3.8-flash). The README provides no documentation URLs for these, so none are listed.